Backend Developer — Step 6

Git, Docker & Deployment

Roadmap objective: Version backend code, containerize it and deploy it safely
Level: Beginner → Intermediate
Last reviewed: September 2026

How to use these notes

Read the concepts first, reproduce the examples yourself, complete the practical lab, and answer the interview questions without looking at the notes. Use the official documentation links as the final authority for changing APIs and platform behavior.

Learning outcomes

- Git workflow
- Branches and commits
- GitHub
- Docker images/containers
- Dockerfile
- Environment variables
- Build vs run
- Deployment checklist

Core concepts

1. Git records source-code history; GitHub hosts repositories and collaboration workflows.

2. Docker packages an application and its dependencies into reproducible containers.

3. Deployment requires more than 'it runs locally': configuration, ports, environment variables, database connectivity, logs and health checks matter.

4. Never commit secrets or production credentials into Git.

Concept map

```
Git, Docker & Deployment
  |
  +--> Learn the concept
  |
  +--> Reproduce a small example
  |
  +--> Apply it in a feature
  |
  +--> Test failure/edge cases
  |
  +--> Document the decision
```

Detailed study guide

1. Git workflow

Study git workflow through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

2. Branches and commits

Study branches and commits through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

3. GitHub

Study github through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

4. Docker images/containers

Study docker images/containers through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

5. Dockerfile

Study dockerfile through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

6. Environment variables

Study environment variables through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

Worked example

Dockerfile + Git workflow

```
FROM node:22-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
EXPOSE 3000
CMD ["node", "server.js"]

# Typical Git flow
git status
git add .
git commit -m "Add API validation"
git push origin main
```

Practice variation: change one input, introduce one failure, predict the result, then run it. Rewrite the example from memory afterward.

Comparison table

Concept	Meaning	Practical use
Git	Version control	Local history/workflow
GitHub	Hosting/collaboration	Remote repository/platform
Docker	Containerization	Reproducible runtime environment

Common mistakes

- Committing .env files
- Huge commits
- Using Docker without understanding ports
- Ignoring logs
- Deploying without production configuration

Best practices

- Keep responsibilities clear and small.
- Validate inputs at system boundaries.
- Prefer readable code over clever code.
- Test important success and failure paths.
- Use official documentation for current API/platform behavior.
- Keep secrets and credentials out of source control.
- Document important trade-offs so another developer can reproduce your reasoning.

Extended practical lab

Complete these tasks in order. The goal is to turn passive knowledge into evidence that you can actually use the topic.

Lab 1

- Create a Git repository and make small commits.
- Write down what you expected, what happened, and why.

Lab 2

- Add a .gitignore and verify secrets are not tracked.
- Write down what you expected, what happened, and why.

Lab 3

- Create a Dockerfile.
- Write down what you expected, what happened, and why.

Lab 4

- Build and run the image locally.
- Write down what you expected, what happened, and why.

Lab 5

- Verify port mapping and environment variables.
- Write down what you expected, what happened, and why.

Lab 6

- Deploy and inspect logs after a deliberate configuration error.
- Write down what you expected, what happened, and why.

Mini-project

Containerize a small Express API with Docker, push the code to GitHub, configure environment variables, and deploy it. Verify logs and health checks.

Acceptance criteria

- A new developer can understand the project from its README.
- The main happy path works from start to finish.
- At least three edge/failure cases are tested.
- The project has meaningful Git commits.
- The project includes a short explanation of design choices.
- If deployment applies, production behavior is verified rather than assumed.

Interview preparation

Practice answering these aloud. A strong answer should define the concept, give a concrete example, mention one trade-off, and explain when you would use it.

Git vs GitHub?

- Answer structure: definition → example → trade-off/limitation → practical use.

Image vs container?

- Answer structure: definition → example → trade-off/limitation → practical use.

Why use .gitignore?

- Answer structure: definition → example → trade-off/limitation → practical use.

What is a Dockerfile?

- Answer structure: definition → example → trade-off/limitation → practical use.

How do environment variables affect deployment?

- Answer structure: definition → example → trade-off/limitation → practical use.

What would you check first when deployment fails?

- Answer structure: definition → example → trade-off/limitation → practical use.

Debugging checklist

- Reproduce the problem consistently.
- Reduce it to the smallest failing example.
- Inspect inputs at each boundary.
- Check the first incorrect value, not only the final error.
- Read the complete error message and stack trace.
- Change one thing at a time.
- Retest the original failure after the fix.
- Add a regression test when the bug is important.

Glossary

Term	Your own definition	Example/use
Repository	Write this in your own words.	Give one project example.
Commit	Write this in your own words.	Give one project example.
Branch	Write this in your own words.	Give one project example.
Remote	Write this in your own words.	Give one project example.
Image	Write this in your own words.	Give one project example.
Container	Write this in your own words.	Give one project example.
Dockerfile	Write this in your own words.	Give one project example.
Port	Write this in your own words.	Give one project example.

Term	Your own definition	Example/use
Environment variable	Write this in your own words.	Give one project example.
Deployment	Write this in your own words.	Give one project example.

Self-test

- Explain Git workflow without using its textbook definition.
- Show a small example of Branches and commits.
- What is the most important boundary in this topic?
- What is one failure mode and how would you diagnose it?
- What trade-off should a developer know before using this technique?
- How would you test the normal path and an edge case?
- How does this topic connect to the next roadmap step?
- What would you change if the system had 100× more users/data?
- What part of this topic would you explain differently to a beginner?
- What can you now build that you could not build before studying this step?

Final checklist

- I can explain and demonstrate Git workflow.
- I can explain and demonstrate Branches and commits.
- I can explain and demonstrate GitHub.
- I can explain and demonstrate Docker images/containers.
- I can explain and demonstrate Dockerfile.
- I can explain and demonstrate Environment variables.
- I can explain and demonstrate Build vs run.
- I can explain and demonstrate Deployment checklist.
- I completed the practical lab.
- I built or extended the mini-project.
- I tested at least three failure/edge cases.
- I can explain one trade-off.
- I can answer the interview questions without notes.
- I know where to find the authoritative documentation.

Recommended documentation

- https://git-scm.com/doc
- https://docs.github.com/
- https://docs.docker.com/
- https://render.com/docs

Recommended videos

- CodeWithHarry — Git and Github Tutorial For Beginners (Full Course)
  https://www.youtube.com/watch?v=AB3J8ufDYHQ (NEW)
- Docker Tutorial - freeCodeCamp (Existing DB resource 105)

Source note: resource references are based on the existing CareerCompass database plus verified web research. For changing APIs, versions, deployment settings and security guidance, consult the linked official documentation before production use.